

Sıranın Üstündeki Kalem

Bilge ve Yonga'nın Maceraları — Buyrukların Dünyası



Prof. Dr. Oğuz Ergin



"Yonga, bilgisayar sayıları nerede tutuyor? Toplama yaparken rakamları nereye koyuyor?" dedi Bilge. Yonga güldü: "Çantayı düşün! Derste her şeyi çantadan çıkarıp çantaya koyamazsın. Sık kullandığın kalemleri sıranın üstüne koyarsın. İşlemcinin de böyle hızlı bir 'sıra üstü' alanı var!"



"Bu hızlı kutulara yazma (register) denir." dedi Yonga. "Yazmalar iřlemcinin tam iindedir. Bellekten yzlerce kat daha hızlıdır. Ama ok kktrler: sadece bir sayı sıar her birine." "Ka tane var?" diye sordu Bilge. "Her iřlemcide sayı deėiřir." dedi Yonga. "RISC-V'te tam olarak 32 tane!"



Yonga yazmaçların görevlerini saydı: • x_0 , adı sıfır: her zaman 0'dır, hiç değişmez. • x_1 , adı ra: dönüş adresini tutar. • x_2 , adı sp: yığıtın tepesini gösterir. • x_5-x_7 , adları t_0-t_2 : geçici değerler için. • $x_{10}-x_{17}$, adları a_0-a_7 : fonksiyona verilen girdiler için. "Hepsinin bir adı ve görevi var." dedi Yonga. "Bir sınıf gibi: her öğrencinin sırası ve görevi!"



"En ilginç yazmaç hangisi?" diye sordu Bilge. "x0!" dedi Yonga. "İçine ne yazarsan yaz, hep sıfır kalır. Asla değişmez. Silinemez. Her zaman sıfırdır." "Ne işe yarar ki?" dedi Bilge. "Çok işe yarar! Bazen 'bir yazmacı sıfırla' ya da 'sıfırla karşılaştı' gibi işlemlere ihtiyaç duyarsın. x0 her zaman orada, hazır bekler." "Çok güçlü bir sıfır!" dedi Bilge.



"Peki ya normal bellek?" diye sordu Bilge. "Yazmalar yetmez mi?" "32 kutu ok az!" dedi Yonga. "Byk programlar milyarlarca sayı saklar. Bunun iin dev bir raf var: bellek (ya da RAM). Her raf gznn bir adresi var, tıpkı sokak numarası gibi." "Raf gz ne kadar byk?" diye sordu Bilge. "Her gzde tam 8 bit sıar. Buna bayt denir."



"Ama bir bayt sadece 0'dan 255'e kadar sayıyı saklar." dedi Yonga. "Daha büyük sayılar için birden fazla bayt kullanırsın. RISC-V'te sayılar genellikle 4 bayt (32 bit) ya da 8 bayt (64 bit) kaplar." "Demek 4 komşu raf gözü birleşip büyük bir sayıyı saklıyor." dedi Bilge. "Harika düşündün! Peki hangisi öne, hangisi arkaya gider? İşte sürpriz burada başlıyor..."



"Dört gözün hepsi aynı büyüklükte, ama parçaların değeri farklı." dedi Yonga. "385'teki 3 üç yüz demek, 5 ise sadece beş." "İkisi de tek basamak ama biri daha değerli!" dedi Bilge. "İşte soru bu: hangisi öne gitsin? Değerlisi öne giderse büyüğü başta, küçüğü öne giderse küçüğü başta deriz. İngilizcesi big-endian ve little-endian." "Ne tuhaf adlar!" dedi Bilge. "Nereden geliyor?" "Güiver'in Gezileri adlı kitaptaki yumurta kavgasından! Yumurta'yı büyük ucundan mı kır, küçük ucundan mı?"



"RISC-V küçüğü başta düzenini kullanır." dedi Yonga. "Demek ki değeri en küçük olan bayt, en düşük adrese gidiyor." "Bunu aklımda nasıl tutayım?" diye sordu Bilge. "Küçüğü başta = değeri en küçük parça önce!" dedi Yonga. "Telefonlar, bilgisayarlar, oyun konsolları gibi çoğu günümüz cihazı küçüğü başta düzenini kullanır." "Demek yumurta kavgasını küçük-ucular kazandı." dedi Bilge gülererek.



"Şimdi anlıyorum." dedi Bilge. "Yazmaçlar çok hızlı ama az. Bellek çok büyük ama daha yavaş." "Tam doğru!" dedi Yonga. "İşlemci önce yazmaçlara bakar. Orada yoksa bellekten alır. Bellekten almak yazmaçtan almaktan yüzlerce kat daha uzun sürer." "Bu yüzden yazmaçları akılcıca kullanmak önemli." dedi Bilge. "Doğru bildin! Buna yazmaç ataması denir. Programı hazırlayan araçların yaptığı zor bir iştir."



"Yığıtın tepesini gösteren yazmacı hatırlıyor musun?" dedi Yonga. "x2, kısa adıyla sp. Yığıt (stack) bir kule gibidir: yalnız en üstüne koyar, yalnız en üstünden alırsın. En son koyduğun, ilk aldığı olur." "Plastik tabak kuleleri gibi!" dedi Bilge. "Alttakini almak için üstündekileri kaldırman gerekir." "Aynen! Kuleye bir tabak koysan da alsan da sp hep en üstü gösterir."



Bilge bir kağıda yazdı: "Diyelim ki x5 yazmacında 10, x6 yazmacında 7 var. Bunları toplamak için işlemci ikisini alır, toplar, sonucu x7 yazmacına yazar." "Aferin!" dedi Yonga. "Ve bu işlemin hiçbir adımında bellekten veri almak gerekmedi. Her şey hızlı yazmaçlarda oldu!"



O akşam Bilge annesiyle yemek masasındayken tuzluęu tutmak istedi. Uzandı, aldı, koydu. Saniyeler içinde. "Tuzluk yazmaç gibi." diye düşündü. "Tam yanımda, çok hızlı." Depo ise çok büyük ama uzakta. Oraya gidip gelmek zaman alırdı. "Bilgisayar da benim gibi düşünüyor." dedi fısıltıyla gülümseyerek.



Bilge yatmadan önce defterine yazdı: • Yazmaç: işlemci içindeki çok hızlı kutu. RISC-V'te 32 tane (x0–x31). • Bellek: devasa bir raf; her göz 1 bayt. • Küçüğü başta: değeri en küçük bayt önce. • Yığıt: plastik tabak kulesi; sp hep en üstte. "Yonga, yavaş yavaş anlıyor gibiyim." dedi Bilge. "Anlıyor gibi değilsin." dedi Yonga sakın bir sesle. "Gerçekten anlıyorsun."

Bugün Ne Öğrendik?

- Yazmaç: İşlemcinin içindeki çok hızlı saklama kutusu. RISC-V'te 32 tane var (x0–x31).
0 x0 yazmacı: Her zaman sıfır değerindedir; yazılamaz.
- Bellek (RAM): Devasa raf deposu. Her göz 1 bayt (8 bit) saklar, her gözün bir adresi var.
- Bayt: 8 bit. Bellekteki en küçük adreslenebilir birim.
- Büyüğü başta / küçüğü başta (big-endian / little-endian): Çok baytlı sayının bellekte nasıl sıralandığı. RISC-V küçüğü başta düzenini kullanır.
- Yığıt (stack): Tabak kulesi gibi çalışan bellek bölgesi. sp yazmacı en üstü gösterir.
- Yazmaç ataması: Programı hazırlayan araçların yazmaçları akıllıca kullanmasına denir.

Deneme Zamanı

1. RISC-V'te kaç yazmaç vardır?
2. x0 yazmacına bir sayı yazarsan ne olur?
3. Bellekte bir gözde kaç bit durur?
4. Yığıtın en üstünü hangi yazmaç gösterir?

Yanıtlar

1. 32 tane; x0'dan x31'e.
2. Değişmez, her zaman sıfır kalır.
3. 8 bit; buna bir bayt denir.
4. sp.

Buyrukların Dünyası



Kitap 3.1a: Dediğimi Yap

Program nedir; kesin, sıralı adımlar ve makinenin tahmin etmemesi



Kitap 3.1b: İki Dünyanın Köprüsü

Buyruk kümesi mimarisi ve Getir-Çöz-Yürüt



>> Kitap 3.2: Sıranın Üstündeki Kalemler

Yazmaçlar, bellek düzeni ve bayt sırası



Kitap 3.3: Zarfın Üstündeki Bölmeler

Buyruk biçimleri ve adresleme kipleri



Kitap 3.4: Parkta Kavşak

Dallanma, döngüler ve altıyordamlar



Kitap 3.5: Mektup Fabrikası

Derleme zinciri: derleyici, çevirici, bağlayıcı



Kitap 3.6: Toplama Makinesi

Aritmetik buyruklar: toplama, çıkarma, çarpma



Kitap 3.7: Sihirli Maskeler

Mantık buyrukları: VE, VEYA, DEĞİL



Kitap 3.8: Sıfırın Gücü

x0 yazmacı ve sıfırın gücü



Kitap 3.9: Kulelerin Oyunu

Yığıt ve altıyordamlar



Kitap 3.10: Taşıyıcılar

Veri aktarma buyrukları: yükle ve sakla



Kitap 3.11: Dedektif Bilge ve Kayıp Sonuç

Hata ayıklama: ara nokta, adım adım yürütme

Bilge ve Yonga'nın Tüm Maceraları

Her kitap tek bir fikri, baştan sona bir hikâyeyle anlatır

Kumdan Bilgisayara

1. Cilt · 13 kitap



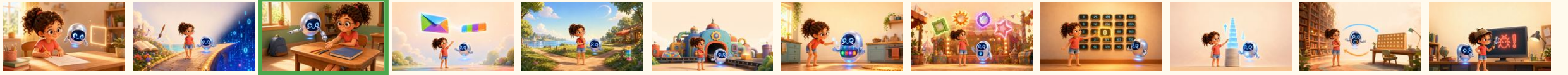
Hız ve Güç

2. Cilt · 10 kitap



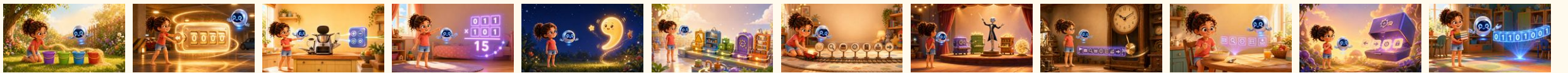
Buyrukların Dünyası

3. Cilt · 12 kitap



İşlemcinin İçi

4. Cilt · 12 kitap



Bütün kitaplar ücretsiz, çevrimiçi:
bilgeveyonga.oguzergin.net

KÜNYE

© 2026 Prof. Dr. Oğuz Ergin

Sıranın Üstündeki Kalem

Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi

Ankara, 2026

Sürüm 1.1, 9 Ağustos 2026

Bu eser, Creative Commons Atıf-GayriTicari-Türetilemez 4.0 Uluslararası (CC BY-NC-ND 4.0) lisansı ile açık erişime sunulmuştur.

Lisans metni: <https://creativecommons.org/licenses/by-nc-nd/4.0/deed.tr>

LİSANSIN KAPSAMADIĞI TÜM HAKLAR SAKLIDIR.

“Bilge ve Yonga” seri adı, “Bilge” ve “Yonga” karakter adları ile figürleri, seri amblemi, marka hakları, eser sahibinin adı ve unvanı ile manevi haklar bu lisansın kapsamı dışındadır ve ayrı yazılı izne bağlıdır.

Metin ve veri madenciliği ile yapay zekâ modeli eğitimi bakımından haklar açıkça saklı tutulmuştur.

Eserler olduğu gibi sunulmaktadır; belirli bir amaca uygunluk konusunda garanti verilmez.

Ticari kullanım izni ve sorularınız için: bilgi@oguzergin.net

Telif ve lisans politikası: bilgeveyonga.oguzergin.net/telif-ve-lisans.html

Atıf: Ergin, O. (2026). Bilge ve Yonga: Bilgisayar Mimarisi Çocuk Kitapları Serisi. <https://bilgeveyonga.oguzergin.net>

Bilge ve Yonga © 2026 Prof. Dr. Oğuz Ergin · CC BY-NC-ND 4.0

Bilge ve Yonga'nın Maceraları

Buyrukların Dünyası

Prof. Dr. Oğuz Ergin

© 2026 · CC BY-NC-ND 4.0